



Micro Interactions

Message ChatGPT



Create image



A

Usage >



Events

0 of 1,000



Links

0 of 25

Usage will reset Jan 5, 2025

Get Dub Pro





poolside is building next generation
AI for software engineering

Concept

Change between states

default → hover

default → active

disabled → enabled

unchecked → checked

invisible → visible

out of view → in to view



HTML can do that?

HTML interactive elements

- `<a>` tag: links to other pages or parts of a page
- input types
 - checkbox (:checked state)
 - radio buttons (:checked state)
- `<button>` tag
 - `type="submit"` for submitting forms
 - `type="button"` for client side interactions with JavaScript





```
<a href="test.html">  
  <button>  
    Click me  
  </button>  
</a>
```

Reminder: please don't do this!

HTML interactive elements

popover API

Make any element a popover

Use for non-blocking UI (e.g., dropdown menus, tooltips, floating panels).

1. Add the **popover** attribute to any element to make it a popover. This will hide it
2. Give it an ID
3. Add the **popovertarget** attribute to any button with the ID of the popover element as the value to open it.
4. Light-dismiss by default: clicking outside the popover element will hide it again.



```
<button class="toggle-popover" popovertarget="myPopover">Toggle Popover</button>
<div class="popover" id="myPopover" popover>
  <p>This is a popover!</p>
</div>
```

Demo: 01-popover

```
<button class="toggle-popover" popovertarget="myPopover">Toggle Popover</button>
<div class="popover" id="myPopover" popover="manual">
  <p>This is a popover!</p>
</div>
```

Demo: 01-popover-manual

HTML interactive elements

popover API

- Beware: popover elements are added to the [top layer](#), so you can't position them absolutely to the parent.
- You need CSS [anchor positioning](#) for that.

Demo: 01-popover-anchor

Advanced example: <https://codepen.io/una/pen/BaGVopL>



HTML interactive elements

<details>: The Details disclosure element

- Consists of a <detail> and <summary> element
- The summary element opens the detail when clicked
- Group them together with the same **name** attribute to open only one at the same time.



```
<details>  
  <summary>Details</summary>  
  Something small enough to escape casual notice.  
</details>
```

Demo: 02-details-summary

```
<details name="reqs">
  <summary>Graduation Requirements</summary>
  <p>
    Requires 40 credits, including a passing grade in health, geography
    history, economics, and wood shop.
  </p>
</details>
<details name="reqs">
  <summary>System Requirements</summary>
  <p>
    Requires a computer running an operating system. The computer must
    memory and ideally some kind of long-term storage. An input device
    as some form of output device is recommended.
  </p>
</details>
<details name="reqs">
  <summary>Job Requirements</summary>
  <p>
    Requires knowledge of HTML, CSS, JavaScript, accessibility, web per
    privacy, security, and internationalization, as well as a dislike o
    broccoli.
  </p>
</details>
```

Demo: 02-details-summary-group

CSS

CSS

- Pseudo-classes or attribute selectors can be used to add or enhance simple interactions
- `:hover`, `:active`, `:focus`, ...
- `:checked`, `:valid`, `:user-valid`, ...
- `[open]`, `:popover-open`
- `:has()`



CSS

Combine pseudo classes with selectors and select other elements

Nested elements:

hover parent → change child/descendant

Next siblings (sister/brother elements: same parent)

check checkbox → change sibling that comes after it

:has() (the “parent” or “god” selector)

changed state of element -> change completely other element



```
<a class="link" href="#">
  hover me
  <svg class="link__arrow" viewBox="0 0 20 20">
    <path d="M4 10H16M16 10L10 4M16 10L10 16" />
  </svg>
</a>
```

```
.link:hover {
  color: deeppink;
}

.link:hover .link__arrow {
  translate: .5rem;
  stroke: deeppink;
}
```

Demo: 03-nested-hover

```
<li class="checkbox">
  <input class="checkbox__input" type="checkbox" name="checkthisout" id="checkthisout">
  <label class="checkbox__label" for="checkthisout">Make me red</label>
  <span class="checkbox__span">And me</span>
</li>
```

```
/* make only the next sibling of the checkbox red when checked */
```

```
.checkbox__input:checked + * {
  color:  #f00;
}
```

```
/* make all siblings of the checkbox red when checked */
```

```
.checkbox__input:checked ~ * {
  color:  #f00;
}
```

Demo: 03-checked-sibling

Combine selectors

```
.checkbox__input:checked~.checkbox__label .checkbox__icon {  
  opacity: 1;  
}
```

Elements with class of checkbox__icon **inside** elements with class of checkbox__label **that come after** elements with class checkbox__input that are checked.

Demo: 03-checked-sibling



<pre> <body> <form class="form"> Thank you <label> <input class="check-me" type="checkbox"> Check me please </label> </form> </body> </pre>	<pre> .form:has(.check-me:checked) .thanks { display: block; } :has(.check-me:checked) { background-color:  hotpink; } </pre>
---	---

Demo: 03-has-checked

CSS

- Know what you're selecting!
- Use CSS variables to simplify selectors
- [Or CSS nesting](#)



```
/* hover state of the link */
.link:hover {
  color: deeppink;
}

/* hover state of the arrow inside the link */
.link:hover .link__arrow {
  translate: .5rem;
  stroke: deeppink;
}
```



```
/* hover state of the link */
.link:hover {
  color: deeppink;

  /* hover state of the arrow inside the link */
  .link__arrow {
    translate: .5rem;
    stroke: deeppink;
  }
}
```

CSS nesting demo: 03-nested-hover-nesting


```

/* initial state of the link */
.link {
  display: inline-flex;
  align-items: center;
  text-decoration: none;
  gap: 0.5rem;
  color: slateblue;
}

/* initial state of the arrow */
.link__arrow {
  display: block;
  width: 1.25rem;
  aspect-ratio: 1;
  stroke: slateblue;
}

/* hover state of the link */
.link:hover {
  color: deeppink;
}

/* hover state of the arrow inside the link */
.link:hover .link__arrow {
  translate: .5rem;
  stroke: deeppink;
}

```



```

/* initial state of the link */
.link {
  display: inline-flex;
  align-items: center;
  text-decoration: none;
  gap: 0.5rem;
  color: slateblue;
}

/* initial state of the arrow */
.link__arrow {
  display: block;
  width: 1.25rem;
  aspect-ratio: 1;
  stroke: var(--link-arrow-stroke, slateblue);
  translate: var(--link-arrow-translate, 0);
}

/* hover state of the link */
.link:hover {
  color: deeppink;

  /* hover state of the arrow inside the link */
  --link-arrow-stroke: deeppink;
  --link-arrow-translate: .5rem;
}

```

CSS variables to simplify selectors. When the parent is hovered, change the variables of its child. Demo: 03-nested-hover-variables

o1-hover-svg

exercise time

Javascript

Javascript

- CSS can't add functionality or change the DOM
- CSS can't listen to events and add more advanced dynamic user interactions.
- It has to make sense semantically: not everything can be a checkbox or a popover...
- Use a `<button type="button">` to add interactivity



Javascript

- Something happens like an event or setInterval
- → change class with classList.
- Combine CSS selectors and select other elements
 - Toggle class of element → change element inside
 - Toggle class → change element after
 - Toggle class → change completely unrelated element with :has()

Demo: 04-toggle-class



Data attributes

Data attributes allow us to store extra information on standard, semantic HTML elements

Starts with **data-** and you give it a custom name

`<element data-custom-name="value">`

Becomes available in javascript as: `dataset.customName`

More info: [MDN](#)



Data attributes

Use them to tie elements together with **data attributes values and id's**.
With JavaScript you can then make them interact with each other.

```
<button class="show-element" data-show-element="element1" type="button">  
    Click me  
</button>
```

```
<div id="element1">  
    Thanks for clicking!  
</div>
```

Demo: 04-data-attributes



Dialog

- The `<dialog>` element is a “modal” element.
- It makes the rest of the page “inert”.
- you cannot interact with the rest of the page while the dialog is open
- Open with `showModal()` and close with `close()`

Demo: 04-dialog



Transitions

Transitions

- Animate the **state change** with transitions
- Set the default transition properties on the **initial state** of **all elements** that need transition. The transition will automatically reverse that way.
- Try to use opacity and transform properties as much as possible.

Demo: 05-nested-hover-transition



Transitions

- transition-property: what properties should transition
- Transition-duration: not too slow, not too fast. Depends on **distance, size, complexity & type of easing**.
- Transition-delay to wait.
- Transition-timing-function:
 - easing
 - the acceleration and deceleration over time (duration)





<https://m2.material.io/design/motion/speed.html#easing>

Transition-timing-function

- linear = no acceleration
- ease-in = start slow, end fast
- ease-out = start fast, end slow
- ease-in-out = start slow, accelerate, end slow
- ease = start a bit slow, accelerate fast, end slow
- cubic-bezier() for **more** complex/pronounced easings
- linear() for **even more** complex/pronounced easings

Demo: 05-transition-timing-functions

Tool: <https://easingwizard.com/> or <https://easings.net/>



Transition discrete properties

- `display: none → display: block`
- `height: 0 → height auto`
- 3 things
 - `interpolate-size: allow-keywords;`
 - `transition-behaviour: allow-discrete`
 - `@starting-style { }`

Demo: 06-details-summary-transition

Demo: 06-popover-transition

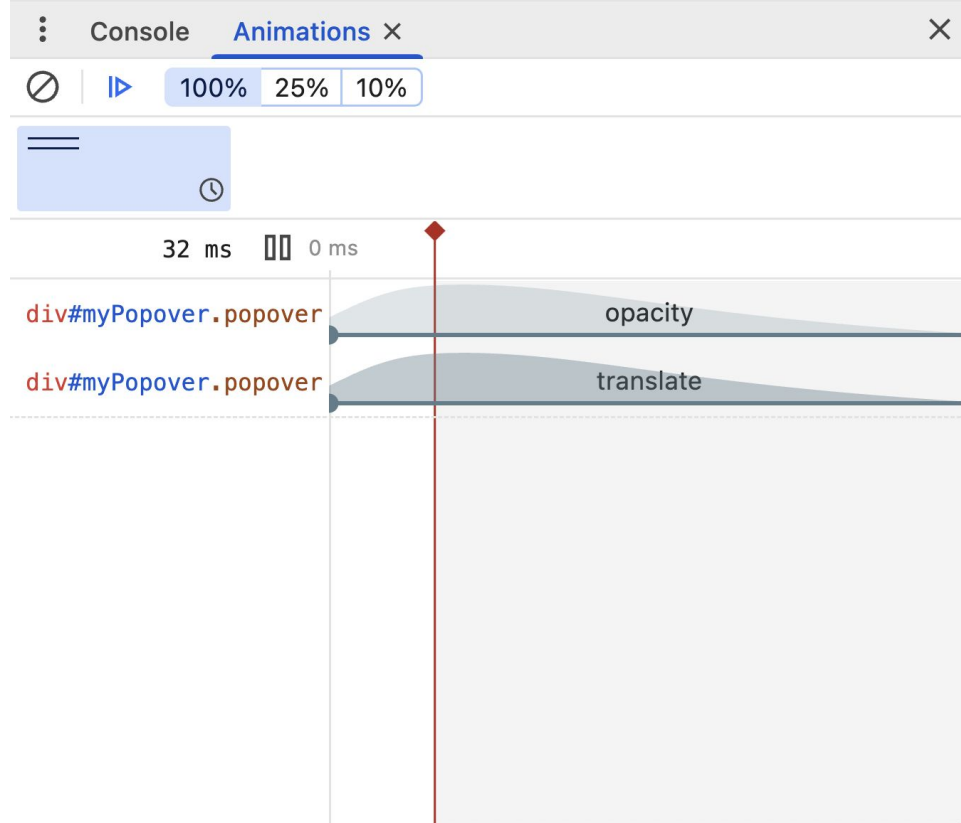
Demo: 06-dialog-transition



Debugging transitions & animations

- Animation panel in dev tools
- Control the speed





Animations panel in dev tools. Let's you control the speed and pause the animation

02-accordion
03-menu
04-copy-me
05-quotes-switcher

exercise time

View Transition API

View Transition API

Want to move elements around in the DOM and animate them?

```
document.startViewTransition(() => updateTheDOMSomehow());
```

Browser takes snapshots of the old and new state and puts it as a layer on top of the whole page inside the `::view-transition` pseudo element.

Demo: 07-view-transition-1



Document x +

localhost:5502/05-micro-interactions/demos/06-view-transition-1/index.html

☆ 📄 🗑️ 📄 New Chrome available

Elements Console Sources >> ⚙️ ⋮ ✕

```
<!DOCTYPE html>
<html lang="en">
  <::view-transition
    <::view-transition-group(root)
      <::view-transition-image-pair(root)
        ...
        <::view-transition-old(root) == $0
        <::view-transition-new(root)
      <head> ... </head>
    <body>
      <div class="box one">...</div>
      <div class="box two">...</div>
      <script src="script.js"></script>
      <!-- Code injected by live-server -->
      <script>...</script>
    </body>
  </html>
```

n-group ::view-transition-image-pair ::view-transition-old

Styles Computed Layout Event Listeners >>

Filter :hov .cls + 🗑️ 📄

```
::view-transition-old(root) {
  inspector-style-sheet:5
}

html::view-transition-old(root) {
  animation-name: -ua-view-transition-fade-out,
  -ua-mix-blend-mode-plus-lighter;
}

html::view-transition-old(*) {
  animation-duration: inherit;
  animation-fill-mode: inherit;
}

::view-transition-old(*) {
  user agent style-sheet
```

1005 x 982

View Transition API

Give it a unique view-transition-name with CSS

```
.ball {  
  view-transition-name: ball;  
}
```

Or with inline style

```
<div class="ball" style="view-transition-name: ball;"></div>
```



Document x +

localhost:5502/05-micro-interactions/demos/06-view-transition-1/index.html

New Chrome available

Elements Console Sources >> ⚙️ ⋮ ✕

```
<!DOCTYPE html>
<html lang="en">
  <::view-transition>
    <::view-transition-group(root)>
      <::view-transition-image-pair(root)>
        <::view-transition-old(root)>
        <::view-transition-new(root)>
      <::view-transition-group(ball)>
        <::view-transition-image-pair(ball)>
          <::view-transition-old(ball) == $0
          <::view-transition-new(ball)>
    <::view-transition-end>
  </::view-transition>
  <head>...</head>
  <body>
    <div class="box one">...</div>
    <div class="box two">...</div>
    <script src="script.js"></script>
    <!-- Code injected by live-server -->
  </body>
</html>
```

⏏

Filter :hov .cls + 🖨️ 📄

```
::view-transition-old(ball) { inspector-stylesheet:8 }
html::view-transition-old(ball) { user agent stylesheet
  animation-name: -ua-view-transition-fade-out,
  -ua-mix-blend-mode-plus-lighter;
}
html::view-transition-old(*) { user agent stylesheet
  animation-duration: inherit;
  animation-fill-mode: inherit;
}
::view-transition-old(*) { user agent stylesheet
```

View Transition API

Customize the animation with CSS

```
::view-transition-group(ball) {  
  animation-duration: 1s;  
  animation-timing-function: var(--elastic-easing);  
}
```



View Transition API

Multiple elements with the same class?

```
.ball {  
  view-transition-class: ball;  
}
```

Each element should have a unique view-transition-name

```
<div class="ball" style="view-transition-name:ball-1;"></div>  
<div class="ball" style="view-transition-name:ball-2;"></div>  
<div class="ball" style="view-transition-name:ball-3;"></div>
```



View Transition API

Multiple elements with the same class?

```
::view-transition-group(.ball) {  
  animation-duration: 1s;  
  animation-timing-function: var(--elastic-easing);  
}
```



Demo: 07-view-transition-2



06-todo-list
07-sticker-wall

exercise time

Examples & inspiration

<https://www.designspells.com/>

<https://www.uilabs.dev/?ref=designerdailyreport.com>

<https://animations.dev/>

<https://ibelick.com/lab>

